

APOSTILA COMPLETA

Controle de Fluxo em Java

switch, for, while, Scanner e 17 exercícios práticos

switch • for • while • do-while • Scanner • Exercícios

7 seções • Loops e Scanner • 17 exercícios de código

Complete Java Masterclass

Sumário

01	Switch Tradicional e Avançado	fall-through, enhanced switch, yield
02	O Laço for	Estrutura, break, continue, números primos
03	while e do-while	Diferenças, quando usar, continue e break
04	Escopo de Variáveis Locais	Blocos, boas práticas, switch vs if
05	Classes, Objetos e static	Conceitos básicos para entender o Scanner
06	Scanner e Exceções	Leitura do console, try-catch, parsing
07	Debugger e Desafios Finais	IntelliJ Debugger, primos, min/max

CAPÍTULO 01

Switch Tradicional e Avançado

Do fall-through ao enhanced switch expression com yield

Por que usar switch?

O switch é uma alternativa elegante a múltiplos if-else quando você precisa testar o valor de uma variável contra várias constantes. É mais legível e, em alguns casos, mais eficiente. Os tipos aceitos são: **byte**, **short**, **int**, **char**, **String** e enums. NÃO aceita **long**, **float**, **double** ou **boolean**.

```
// Tradicional (colon : + break):

switch (diaSemana) {

    case 1: System.out.println("Segunda"); break;

    case 2: System.out.println("Terça"); break;

    default: System.out.println("Outro"); break;

}


// Enhanced switch EXPRESSION (Java 14+ – recomendado):

String nomeDia = switch (diaSemana) {

    case 0 -> "Domingo";

    case 1 -> "Segunda";

    case 6 -> "Sábado";

    default -> "Dia inválido";

};
```

```
// yield – quando o case precisa de múltiplas linhas:

String resultado = switch (valor) {

    case 1 -> "Simples";

    case 2 -> {

        String msg = "Calculando...";

        System.out.println(msg);

        yield "Calculado"; // yield retorna o valor do bloco

    }

    default -> "Desconhecido";

};
```

■ ■ **Atenção:** Fall-through: sem break, a execução cai para o próximo case automaticamente. Isso pode ser intencional (agrupar cases com mesmo comportamento) ou acidental (bug). No enhanced switch com ->, não existe fall-through.

CAPÍTULO 02

O Laço for

Estrutura, break, continue e aplicações práticas

Anatomia do for

O for é a estrutura de loop mais usada quando você sabe antecipadamente quantas iterações serão feitas. Ele tem três partes separadas por ponto e vírgula: inicialização (executa uma vez antes do loop), condição (verificada antes de cada iteração — quando false, o loop para) e incremento (executa após cada iteração).

```
// Estrutura básica:

for (int i = 0; i < 5; i++) {

    System.out.println("i = " + i); // 0, 1, 2, 3, 4

}

// break – sai imediatamente do loop:

for (int i = 0; i < 100; i++) {

    if (i == 5) break;

    System.out.println(i); // imprime 0, 1, 2, 3, 4

}

// continue – pula para a próxima iteração:

for (int i = 0; i <= 10; i++) {

    if (i % 2 != 0) continue; // pula ímpares

    System.out.println(i); // imprime 0, 2, 4, 6, 8, 10
```

```
}
```

CAPÍTULO 03

while e do-while

Loops baseados em condição e a diferença fundamental

A Diferença Fundamental

Ambos os loops repetem código baseado em uma condição, mas diferem em QUANDO a condição é verificada. O **while** verifica a condição ANTES de executar o bloco — pode nunca executar se a condição for false desde o início (0 ou mais execuções). O **do-while** executa o bloco PRIMEIRO e depois verifica a condição — executa sempre pelo menos uma vez (1 ou mais execuções).

```
// while – verifica antes:

int n = 0;

while (n < 15) {

    n++;

    if (n <= 5) continue; // pula os 5 primeiros

    if (n >= 10) break; // para ao chegar em 10

    System.out.println(n); // imprime 6, 7, 8, 9

}


// do-while – executa ao menos uma vez (menu):

int opcao;

do {

    System.out.println("1-Iniciar 2-Config 0-Sair");

    opcao = scanner.nextInt();
```

```
processarOpcao(opcao);  
  
} while (opcao != 0); // note o ; obrigatório após while!
```

■ **Regra:** Prefira for quando souber quantas iterações haverá. Prefira while/do-while quando o número de iterações depender de uma condição dinâmica. Use do-while quando precisar executar pelo menos uma vez (menus, validações).

CAPÍTULO 04

Escopo de Variáveis Locais

Acessibilidade de variáveis dentro de blocos

O que é Escopo?

Escopo é a acessibilidade de uma variável em um dado ponto do código. Uma variável local existe apenas dentro do bloco onde foi declarada (e nos blocos aninhados dentro dele). Ao sair do bloco, a variável deixa de existir — tentar usá-la fora causa erro de compilação.

```
for (int i = 0; i < 5; i++) {  
  
    System.out.println(i); // i em escopo aqui  
  
}  
  
// System.out.println(i); // ■ ERRO: i fora de escopo!  
  
// Diferença entre if e switch:  
  
// if: cada bloco {} tem escopo próprio  
  
if (x > 0) { int v = 1; }  
  
// else { v++; } // ■ ERRO: v não existe aqui  
  
// switch tradicional sem {}: variável pode vazar entre cases  
  
switch (x) {  
  
    case 1: int v = 1; // sem break – vaza!  
  
    case 2: v++; // funciona (mas é confuso!)  
  
}
```

CAPÍTULO 05

Classes, Objetos e static

Conceitos fundamentais para entender e usar o Scanner

Classe vs. Objeto

Uma classe é um tipo de dado customizado — como um molde ou formulário em branco. Um objeto é uma instância da classe — o formulário preenchido. Você cria objetos com a palavra-chave **new**. Campos **static** pertencem à classe e são compartilhados por todas as instâncias. Campos de instância pertencem a cada objeto individualmente.

```
// Criando um objeto Scanner para ler do console:

import java.util.Scanner;

Scanner scanner = new Scanner(System.in);

// Métodos estáticos chamados via NomeDaClasse:

int max = Integer.MAX_VALUE; // campo static

double pi = Math.PI; // campo static

int abs = Math.abs(-42); // método static → 42

int num = Integer.parseInt("123"); // método static → 123
```

CAPÍTULO 06

Scanner e Tratamento de Exceções

Leitura segura do console com try-catch e parsing

Usando o Scanner

A classe **Scanner** da biblioteca padrão Java permite ler dados do console. Para leitura de texto completo (incluindo espaços), use **nextLine()**. Para converter a string recebida para um número, use métodos estáticos como **Integer.parseInt()** ou **Double.parseDouble()**. Se a entrada não for um número válido, esses métodos lançam **NumberFormatException**.

```
import java.util.Scanner;

Scanner scanner = new Scanner(System.in);

int total = 0, contagem = 0;

while (contagem < 5) {

    System.out.print("Digite o número #" + (contagem+1) + ": ");

    String entrada = scanner.nextLine();

    try {

        int numero = Integer.parseInt(entrada);

        total += numero;

        contagem++;

    } catch (NumberFormatException e) {

        System.out.println("Número inválido! Tente novamente.");

    }

}
```

```
System.out.println("Soma: " + total);
```

CAPÍTULO 07

Debugger e Desafios Finais

IntelliJ Debugger e os algoritmos clássicos

IntelliJ Debugger

O debugger do IntelliJ permite inspecionar o código linha por linha durante a execução. Adicione um breakpoint clicando na margem esquerda da linha. Use F8 (Step Over) para avançar linha a linha, F7 (Step Into) para entrar num método, e F9 (Resume) para continuar até o próximo breakpoint.

Desafio: Números Primos

```
public static boolean isPrime(int n) {  
  
    if (n < 2) return false;  
  
    // Só verifica até a raiz quadrada – muito mais eficiente!  
  
    for (int i = 2; i <= Math.sqrt(n); i++) {  
  
        if (n % i == 0) return false;  
  
    }  
  
    return true;  
  
}
```

Algoritmo: Soma de Dígitos

```
// Extraíndo dígitos com % 10 e / 10:  
  
// Exemplo: 1234  
  
// 1234 % 10 = 4 → soma += 4, n = 1234/10 = 123  
  
// 123 % 10 = 3 → soma += 3, n = 123/10 = 12
```

```
// ...continue até n < 10
```

Parabéns por concluir a apostila!

Com o domínio de switch, loops e Scanner, você pode criar programas interativos completos que leem dados do usuário, processam com lógica e exibem resultados — a base de qualquer aplicação real.

Complete Java Masterclass